

Functional GPU Programming

Conor Hoekstra



code_report

|

codereport



Array GPU Programming

Conor Hoekstra

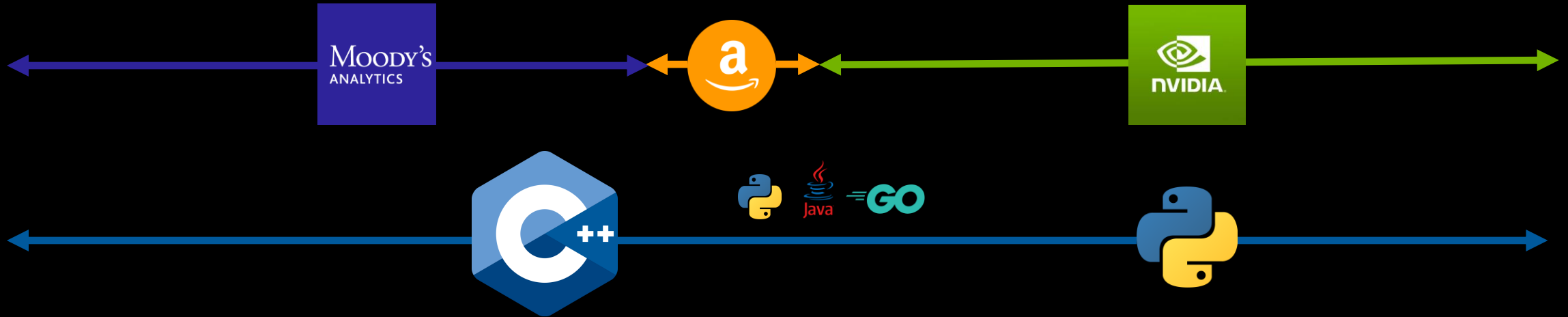
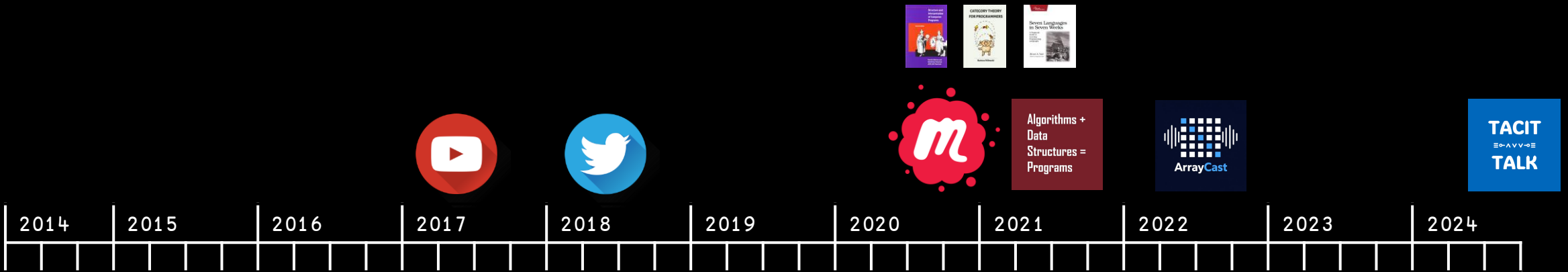


code_report



codereport





About Me

Conor Hoekstra / @code_report





401 Videos



52 (33) Talks

Algorithms +
Data
Structures =
Programs

282 Episodes
@adspthepodcast



123 Episodes
@arraycast



TACIT
≡ ∅ ∧ ∨ ∨ ∅ ≡
TALK

35 Episodes
@codereport





Home



Shorts



Subscriptions



You



code_report

@code_report · 62.5K subscribers · 401 videos

Welcome to the code_report YouTube channel. ...more

conorshakory.com and 6 more links

Customize channel

Manage videos

Home

Videos

Live

Podcasts

Courses

Playlists

Posts



Latest

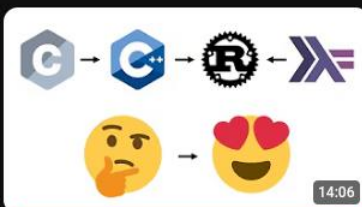
Popular

Oldest



1 Problem, 24 Programming Languages

383K views · 3 years ago



From C C++ Rust

182K views · 3 years ago



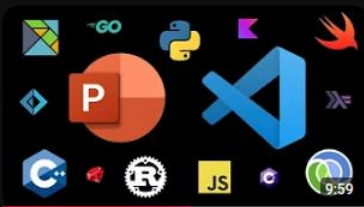
1 Problem, 16 Programming Languages
(C++ vs Rust vs Haskell vs Python vs APL...)

162K views · 4 years ago



Functional vs Array Programming

142K views · 4 years ago



How to Make Beautiful Code Presentations

136K views · 3 years ago



Difference between HackerRank, LeetCode,
topcoder and Codeforces

111K views · 8 years ago



1 Problem, 8 Programming Languages (C++
vs Rust vs Haskell vs Clojure...)

111K views · 5 years ago



1 Problem, 6 Programming Languages (C++
vs Rust vs Haskell vs APL vs Clojure vs...)

101K views · 5 years ago



conorshakory.com

Conor Hoekstra

aka code_report

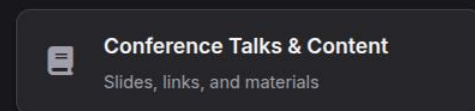
Research Scientist at NVIDIA

(Full Experience)

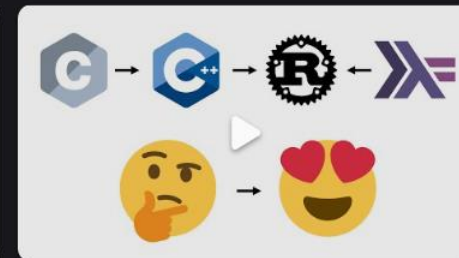
Research Scientist at NVIDIA working on array programming models and languages. Creator of the [Parrot](#) library (a high-level, array-based library with implicit GPU fusion). Extremely passionate about programming languages, algorithms, and beautiful code. Founder and organizer of the [Programming Languages Virtual Meetup](#). Host of four podcasts and an avid conference speaker.



PODCASTS



YOUTUBE



CONFERENCE TALKS



github.com/codereport/Content

Array GPU Programming

Conor Hoekstra



code_report



codereport







Parrot



Parrot

A high level, parallel, array-based library
with implicit fusion



Parrot

<https://github.com/nvlabs/parrot>



Parrot

<https://github.com/nvlabs/parrot>
Apache 2.0 License

Show Docs / GitHub

"Great stuff, I've been waiting for this library ... it's a game changer"

"Wow what a beauty! I thought that was
rust for a second"

"Exquisite!"

"Beautiful chained expressions wow"

"Wow! The parrot example blew me away - it's C++ yet so ergonomic!"

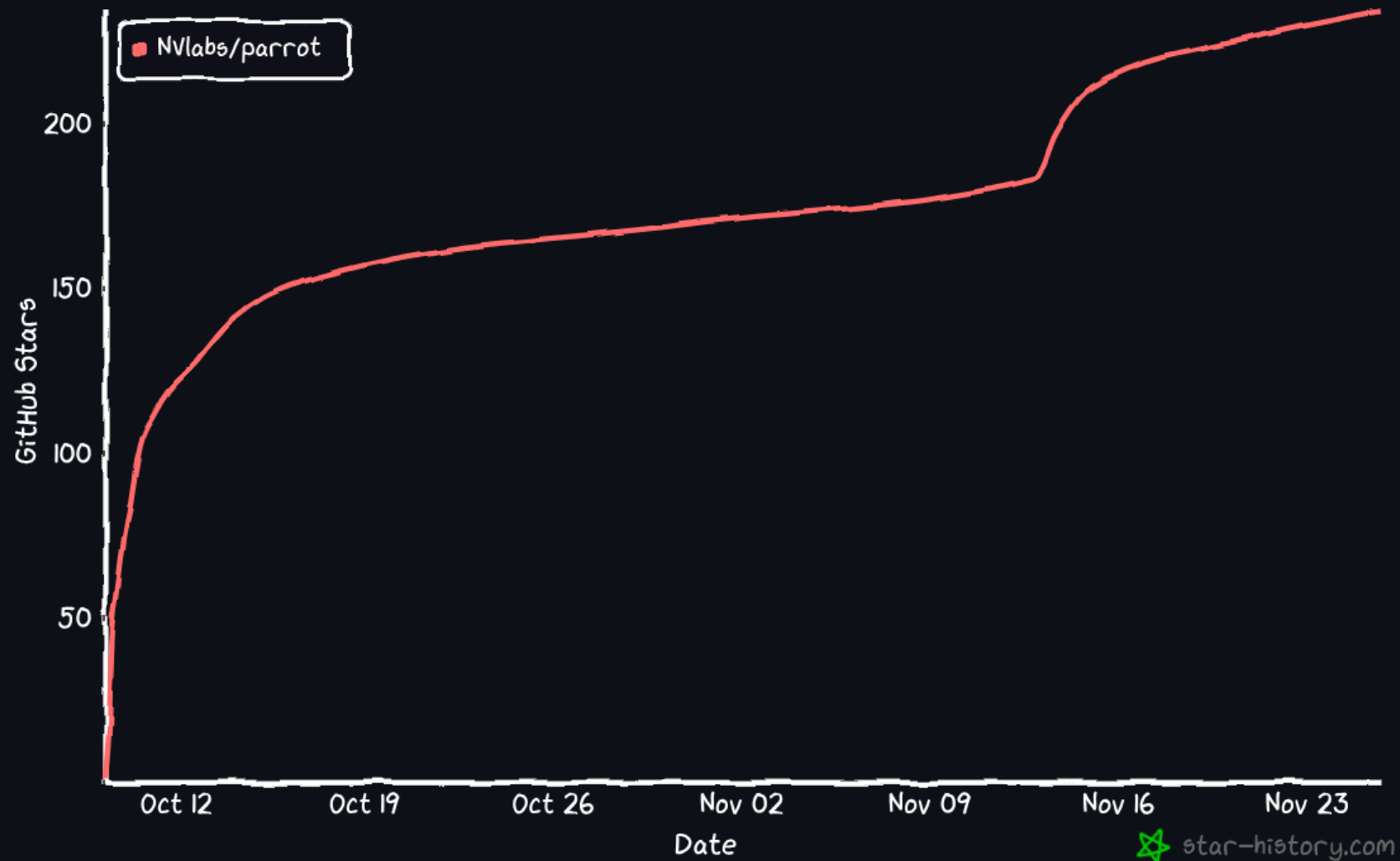
"Parrot's ... solution is so nice compared to thrust"

"First C++ code I can comprehend 😄 "

"I thought it [the parrot] was duolingo"



Star History



Overview

Problem 1



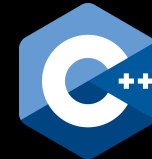
API Overview



Problem 2



Problem 3



Problem 4



Sum of Squares (SOS)





```
#include <algorithm>
#include <print>
#include <ranges>

auto sos(int N) {
    return std::ranges::fold_left(
        std::ranges::views::iota(0, N) |
        std::ranges::views::transform([](int x) { return x * x; }),
        0,
        std::plus{});
}

int main() { std::print("{} ", sos(10)); }
```





```
#include <thrust/device_vector.h>
#include <thrust/sequence.h>
#include <thrust/transform_reduce.h>
#include <iostream>

struct square {
    __host__ __device__ int operator()(const int& x) const { return x * x; }
};

auto sos(int N) {
    thrust::device_vector<int> d_vec(N);
    thrust::sequence(d_vec.begin(), d_vec.end());
    return thrust::transform_reduce(
        d_vec.begin(), d_vec.end(), square(), 0, thrust::plus<int>());
}

int main() {
    auto result = sos(10);
    std::cout << result << std::endl;
    return 0;
}
```



```
#include <thrust/device_vector.h>
#include <thrust/sequence.h>
#include <thrust/transform_reduce.h>
#include <iostream>

auto sos(int N) -> int {
    thrust::device_vector<int> d_vec(N);
    thrust::sequence(d_vec.begin(), d_vec.end());
    return thrust::transform_reduce(
        d_vec.begin(),
        d_vec.end(),
        [] __host__ __device__(int x) { return x * x; },
        0,
        thrust::plus<int>());
}

int main() {
    auto result = sos(10);
    std::cout << result << std::endl;
    return 0;
}
```



```
#include <thrust/device_vector.h>
#include <thrust/sequence.h>
#include <thrust/transform_reduce.h>
#include <iostream>

auto sos(int N) -> int {
    auto iota = thrust::counting_iterator<int>(0);
    return thrust::transform_reduce(
        iota,
        iota + N,
        [] __host__ __device__(int x) { return x * x; },
        0,
        thrust::plus<int>());
}

int main() {
    auto result = sos(10);
    std::cout << result << std::endl;
    return 0;
}
```




```
#include <thrust/iterator/counting_iterator.h>
#include <thrust/iterator/transform_iterator.h>
#include <thrust/reduce.h>
#include <iostream>

auto sos(int N) -> int {
    auto iota = thrust::counting_iterator<int>(0);
    auto map = thrust::make_transform_iterator(
        iota, [] __host__ __device__(int x) { return x * x; });
    return thrust::reduce(map, map + N, 0);
}

int main() {
    auto result = sos(10);
    std::cout << result << std::endl;
    return 0;
}
```





```
auto sos(int N) {  
  
}
```



```
auto sos(int N) {  
    return parrot::range(N)  
}
```



```
auto sos(int N) {  
    return parrot::range(N)  
        .sq()  
}
```



```
auto sos(int N) {  
    return parrot::range(N)  
        .sq()  
        .sum();  
}
```



```
auto sos(int N) {  
    return parrot::range(N).print()  
        .sq().print()  
        .sum().print();  
}
```



```
#include "parrot.hpp"
```

```
auto sos(int N) {  
    return parrot::range(N)  
        .sq()  
        .sum();  
}
```

```
int main() { sos(10).print(); }
```




Parrot API

Fused
1-index Maps (Unary)
★ [map](#)
 [abs](#)
 [dble](#)
 [enumerate](#)
 [even](#)
 [half](#)
 [log](#)
 [exp](#)
 [neg](#)
 [odd](#)
 [rand](#)
 [sign](#)
 [sq](#)
 [sqrt](#)
1-index Maps (Binary)
★ [map2](#)
 [add \(+\)](#)
 [div \(/\)](#)
 [gt \(>\)](#)
 [gte \(>=\)](#)
 [idiv](#)
 [lt \(<\)](#)
 [lte \(<=\)](#)
 [max](#)
 [min](#)
 [minus \(-\)](#)
 [times \(*\)](#)
 [eq \(==\)](#)
 [pairs](#)

2-index Maps
★ [map_adj](#)
 [deltas](#)
 [differ](#)
 Joins
 [append](#)
 [prepend](#)
Products
 [cross](#)
 [outer](#)
Reshapes
 [take](#)
 [drop](#)
 [transpose](#)
 [reshape](#)
 [cycle](#)
 [repeat](#)
Copying
 [replicate](#)
Permutations
 [rev](#)
 [gather](#)
Conditionally Fused
Compactions
★ [keep](#)
 [filter](#)
 [where](#)
 [uniq](#)
 [distinct](#)

Materializing
Reductions
★ [reduce](#)
 [all](#)
 [any](#)
 [maxr](#)
 [max_by_key](#)
 [minr](#)
 [minmax](#)
 [prod](#)
 [sum](#)
Scans
★ [scan](#)
 [alls](#)
 [anys](#)
 [maxs](#)
 [mins](#)
 [prods](#)
 [sums](#)
Permutations
 [sort](#)
 [sort_by](#)
 [sort_by_key](#)
Compactions
 [rle](#)
Copying
 [replicate](#)
Split-Reductions
 [chunk_by_reduce](#)
Comparisons
 [match](#)

Properties
 [size](#)
 [rank](#)
 [shape](#)
Accessors
 [value](#)
 [front](#)
 [back](#)
 [to_host](#)
Array Creation
 [array](#)
 [range](#)
 [scalar](#)
 [matrix](#)
I/O
 [print](#)
Function Objects
 Accessors
 [fst](#)
 [snd](#)
 Binary Operations
 [eq](#)
 [gt](#)
 [gte](#)
 [lt](#)
 [lte](#)
 [max](#)
 [min](#)
 [mul](#)
 [add](#)

Fused
1-index Maps (Unary)

 map
abs
db1e
enumerate
even
half
log
exp
neg
odd
rand
sign
sq
sqrt

Fused
1-index Maps (Binary)

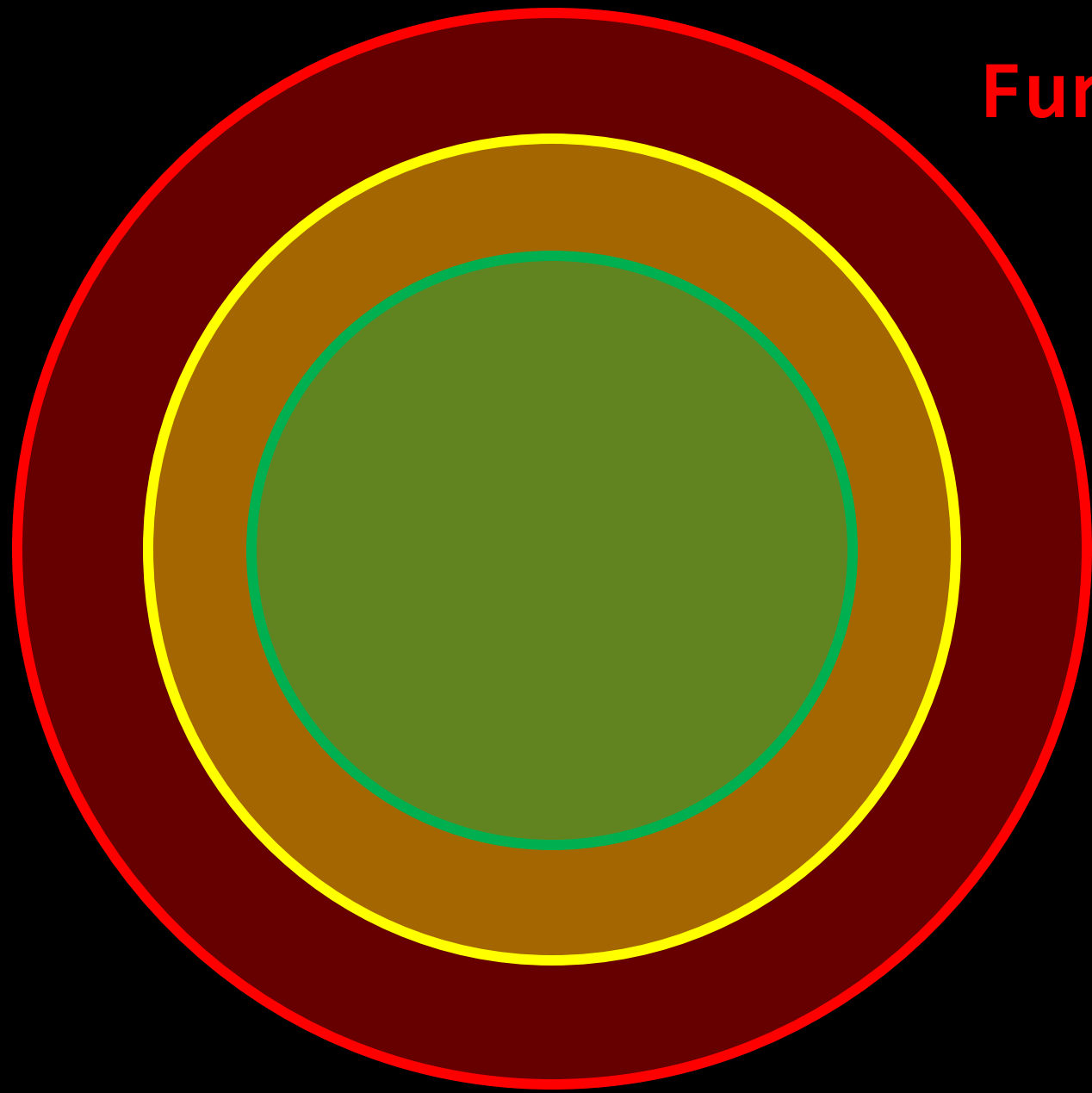
 map2
add (+)
div (/)
gt (>)
gte (>=)
idiv
lt (<)
lte (<=)
max
min
minus (-)
times (*)
eq (==)
pairs

Functional vs Array Programming

**Array = Functional
+ Rank Polymorphism**

**Array* = Functional
+ Rank Polymorphism
+ Symbols**

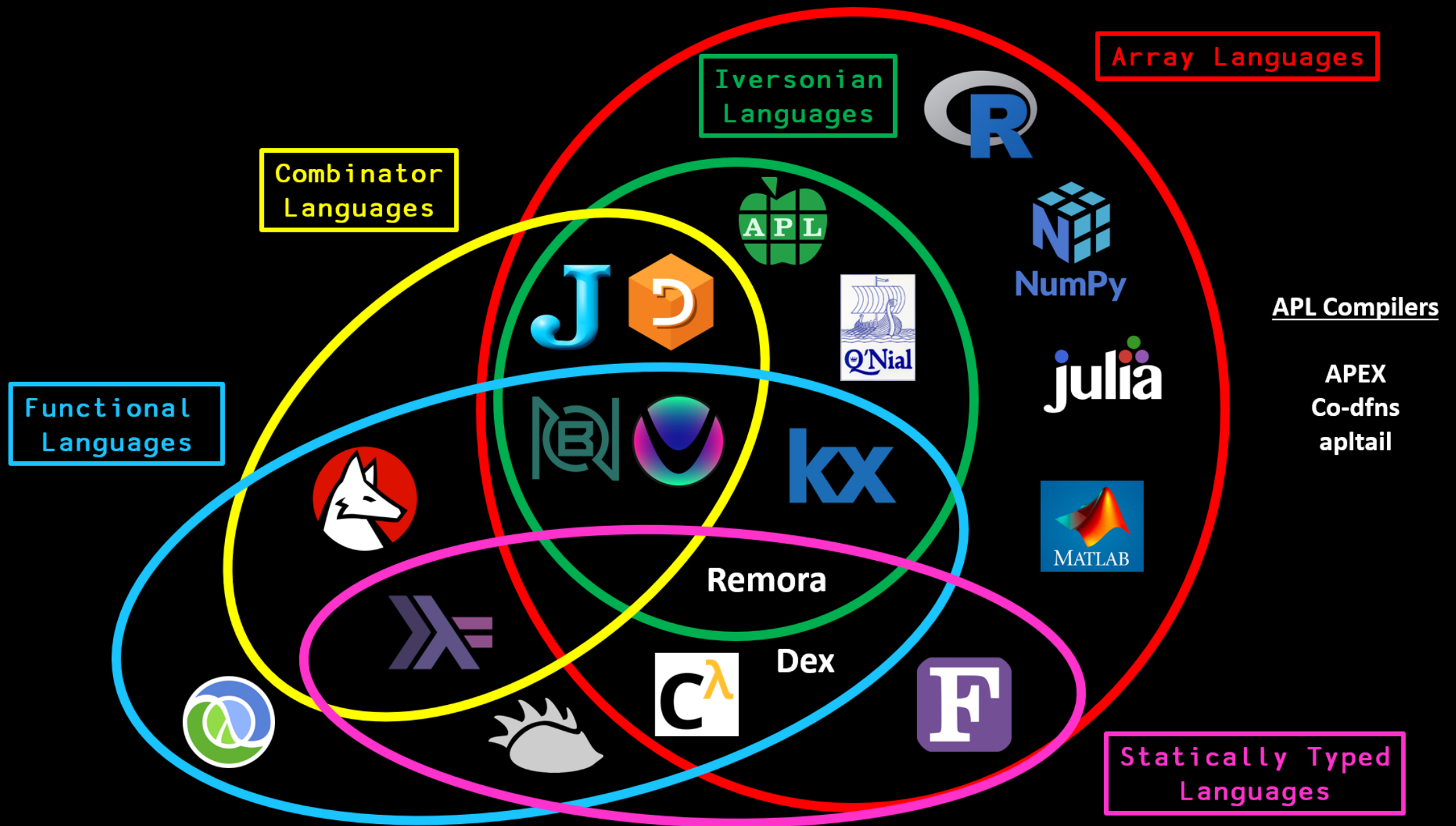
* Iversonian (APL, BQN, Uua)



Functional

Array

**Iversionian
Array**





```
auto add_one(auto& matrix) {  
    for (auto& row : matrix) {  
        for (auto& e : row) {  
            e += 1;  
        }  
    }  
    return matrix;  
}
```



```
auto add_one(const auto& matrix) {  
    return matrix | std::ranges::views::transform([](const auto& row) {  
        return row | std::ranges::views::transform(  
            [](auto e) { return e + 1; });  
    });  
}
```



```
auto add_one(auto matrix) {  
    return matrix + 1;  
}
```



```
auto add_one(auto matrix) {  
    return matrix.add(1);  
}
```



```
auto add_one(auto matrix) {  
    return matrix + 1;  
}
```

Fused 1-index Maps (Unary)

 map
abs
db1e
enumerate
even
half
log
exp
neg
odd
rand
sign
sq
sqrt

Fused 1-index Maps (Binary)

 map2
add (+)
div (/)
gt (>)
gte (>=)
idiv
lt (<)
lte (<=)
max
min
minus (-)
times (*)
eq (==)
pairs

Fused

1-index Maps (Unary)

★ [map](#)
[abs](#)
[dble](#)
[enumerate](#)
[even](#)
[half](#)
[log](#)
[exp](#)
[neg](#)
[odd](#)
[rand](#)
[sign](#)
[sq](#)
[sqrt](#)

1-index Maps (Binary)

★ [map2](#)
[add \(+\)](#)
[div \(/\)](#)
[gt \(>\)](#)
[gte \(>=\)](#)
[idiv](#)
[lt \(<\)](#)
[lte \(<=\)](#)
[max](#)
[min](#)
[minus \(-\)](#)
[times \(*\)](#)
[eq \(==\)](#)
[pairs](#)

2-index Maps

★ [map_adj](#)
[deltas](#)
[differ](#)
Joins
[append](#)
[prepend](#)
Products
[cross](#)
[outer](#)
Reshapes
[take](#)
[drop](#)
[transpose](#)
[reshape](#)
[cycle](#)
[repeat](#)
Copying
[replicate](#)
Permutations
[rev](#)
[gather](#)

Conditionally Fused

Compactions
★ [keep](#)
[filter](#)
[where](#)
[uniq](#)
[distinct](#)

Materializing

Reductions

★ [reduce](#)
[all](#)
[any](#)
[maxr](#)
[max_by_key](#)
[minr](#)
[minmax](#)
[prod](#)
[sum](#)
Scans
★ [scan](#)
[alls](#)
[anys](#)
[maxs](#)
[mins](#)
[prods](#)
[sums](#)

Permutations

[sort](#)
[sort_by](#)
[sort_by_key](#)

Compactions

[rle](#)

Copying
[replicate](#)

Split-Reductions

[chunk_by_reduce](#)

Comparisons

[match](#)

Properties

[size](#)
[rank](#)
[shape](#)

Accessors

[value](#)
[front](#)
[back](#)
[to_host](#)

Array Creation

[array](#)
[range](#)
[scalar](#)
[matrix](#)

I/O

[print](#)

Function Objects

Accessors

[fst](#)
[snd](#)

Binary Operations

[eq](#)
[gt](#)
[gte](#)
[lt](#)
[lte](#)
[max](#)
[min](#)
[mul](#)
[add](#)

Table 1: Parrot method mapping (core transforms, masking, and reductions).

Parrot (C++)	Parrot Python	CUDA C++ backend	cuda.compute backend
map(op)	map(op)	thrust::make_transform_iterator	iterators.TransformIterator
map2(arr, op)	map2(arr, op)	thrust::make_zip_iterator + thrust::make_transform_iterator	iterators.ZipIterator + iterators.TransformIterator
map2(scalar, op)	map2(scalar, op)	thrust::make_transform_iterator	clone + map(bound_op)
times(arg)	times(arg)	 map2(arg,mul)	 map2(arg,mul_op)
add(arg)	add(arg)	 map2(arg,add)	 map2(arg,add_op)
minus(arg)	minus(arg)	 map2(arg,minus)	 map2(arg,sub_op)
min(arg)	min(arg)	 map2(arg,min)	 map2(arg,min_op)
max(arg)	max(arg)	 map2(arg,max)	 map2(arg,max_op)
div(arg)	div(arg)	 map2(arg,div)	 map2(arg,div_op)
mod(arg)	mod(arg)	 map2(arg,mod)	 map2(arg,mod_op)
idiv(arg)	idiv(arg)	 map2(arg,idiv)	 map2(arg,idiv_op)
as<T>	astype(dtype)	 map(cast_functor)	 map(cast_op)
gte(arg)	gte(arg)	 map2(arg,gte)	 map2(arg,gte_op)
lte(arg)	lte(arg)	 map2(arg,lte)	 map2(arg,lte_op)
gt(arg)	gt(arg)	 map2(arg,gt)	 map2(arg,gt_op)
lt(arg)	lt(arg)	 map2(arg,lt)	 map2(arg,lt_op)
eq(arg)	eq(arg)	 map2(arg,eq)	 map2(arg,eq_op)
neq(arg)	neq(arg)	 map2(arg,neq)	 map2(arg,neq_op)
double()	double()	 map(double_functor)	 map(double_op)
half()	half()	 map(half_functor)	 map(half_op)
abs()	abs()	 map(abs_functor)	 map(abs_op)
log()	log()	 map(log_functor)	 map(log_op)
exp()	exp()	 map(exp_functor)	 map(exp_op)
neg()	neg()	 map(negate)	 map(neg_op)
odd()	odd()	 map(odd_functor)	 map(odd_op)
even()	even()	 map(even_functor)	 map(even_op)
sign()	sign()	 map(sign_functor)	 map(sign_op)
sqrt()	sqrt()	 map(sqrt_functor)	 map(sqrt_op)
sq()	sq()	 map(sq_functor)	 map(square_op)
rand()	rand()	zip(index,value) + transform random functor	ZipIterator + TransformIterator random op
map_adj(op)	map_adj(op)	thrust::make_zip_iterator + thrust::make_transform_iterator (zip_function)	PermutationIterator + ZipIterator + TransformIterator
differ()	differ()	 map_adj(neq)	 map_adj(neq_op)
deltas()	deltas()	 map_adj(delta)	 map_adj(delta_op)
keep(mask)	keep(mask)	mask-aware fusion_array constructor	lazy mask in ParrotArray (_mask)
filter(pred)	filter(predicate)	thrust::make_transform_iterator +  keep	algorithms.select
where()	where()	 range(size).keep(this)	 range(length).keep(self)
gather(indices)	gather(indices)	thrust::make_permutation_iterator	iterators.PermutationIterator
reduce<O>(init, op)	reduce(op, init, axis=0)	thrust::reduce	algorithms.reduce_into

Problem 2

(taken from PaddlePaddle)



```
// https://github.com/PaddlePaddle/Paddle/blob/80f1123eb0c...

template <typename T>
static void Get_____(
    const phi::GPUContext& dev_ctx,
    const DenseTensor* input_tensor,
    const int64_t num_cols,
    const int64_t num_rows,
    T* out_tensor,
    int64_t* indices_tensor)
{
    DenseTensor input_tmp;
    input_tmp.Resize(common::make_ddim({num_rows, num_cols}));
    T* input_tmp_data = dev_ctx.Alloc<T>(&input_tmp);
    phi::Copy(dev_ctx, *input_tensor, dev_ctx.GetPlace(), false, &input_tmp);

    thrust::device_ptr<T> out_tensor_ptr(out_tensor);
    thrust::device_ptr<int64_t> indices_tensor_ptr(indices_tensor);

    for (int64_t i = 0; i < num_rows; ++i) {
        T* begin = input_tmp_data + num_cols * i;
        T* end = input_tmp_data + num_cols * (i + 1);
        thrust::device_vector<int64_t> indices_data(num_cols);
        thrust::sequence(thrust::device,
            indices_data.begin(),
            indices_data.begin() + num_cols);
        thrust::sort_by_key(thrust::device, begin, end, indices_data.begin());
        int unique = 1 + thrust::inner_product(thrust::device,
            begin,
            end - 1,
            begin + 1,
            0,
            thrust::plus<int>(),
            thrust::not_equal_to<T>());
        thrust::device_vector<T> keys_data(unique);
        thrust::device_vector<int64_t> cnts_data(unique);
        thrust::reduce_by_key(thrust::device,
            begin,
            end,
            thrust::constant_iterator<int>(1),
            keys_data.begin(),
            cnts_data.begin());
        auto it = thrust::max_element(
            thrust::device, cnts_data.begin(), cnts_data.begin() + unique);
        T ____ = keys_data[it - cnts_data.begin()];
        int64_t counts = cnts_data[it - cnts_data.begin()];
        auto pos = thrust::find(thrust::device, begin, end, mode);
        int64_t index = indices_data[pos - begin + counts - 1];
        out_tensor_ptr[i] = static_cast<T>(mode);
        indices_tensor_ptr[i] = static_cast<int64_t>(index);
    }
}
```



```
// https://github.com/PaddlePaddle/Paddle/blob/80f1123eb0c...
```

```
template <typename T>
static void Get_____(/* ... */) {
    // initialization

    for (int64_t i = 0; i < num_rows; ++i) {
        T* begin = input_tmp_data + num_cols * i;
        T* end = input_tmp_data + num_cols * (i + 1);
        thrust::device_vector<int64_t> indices_data(num_cols);
        thrust::sequence(thrust::device,
            indices_data.begin(),
            indices_data.begin() + num_cols);
        thrust::sort_by_key(thrust::device, begin, end, indices_data.begin());
        int unique = 1 + thrust::inner_product(thrust::device,
            begin,
            end - 1,
            begin + 1,
            0,
            thrust::plus<int>(),
            thrust::not_equal_to<T>());
        thrust::device_vector<T> keys_data(unique);
        thrust::device_vector<int64_t> cnts_data(unique);
        thrust::reduce_by_key(thrust::device,
            begin,
            end,
            thrust::constant_iterator<int>(1),
            keys_data.begin(),
            cnts_data.begin());
        auto it = thrust::max_element(
            thrust::device, cnts_data.begin(), cnts_data.begin() + unique);
        T ____ = keys_data[it - cnts_data.begin()];
        int64_t counts = cnts_data[it - cnts_data.begin()];
        auto pos = thrust::find(thrust::device, begin, end, mode);
        int64_t index = indices_data[pos - begin + counts - 1];
        out_tensor_ptr[i] = static_cast<T>(mode);
        indices_tensor_ptr[i] = static_cast<int64_t>(index);
    }
}
```



```
int unique = 1 + thrust::inner_product(
    begin,
    end - 1,
    begin + 1,
    0,
    thrust::plus<int>(),
    thrust::not_equal_to<T>());
```



```
int unique = 1 + thrust::inner_product(  
    begin,  
    end - 1,  
    begin + 1,  
    0,  
    thrust::plus<int>(),  
    thrust::not_equal_to<T>());
```



```
int unique = thrust::unique_count(begin, end);
```

[[digression]]

[illegible]



```
template <typename I>
auto unique_count(I first, I last) {
    auto zip = thrust::make_zip_iterator(first, first + 1);
    auto map = thrust::make_transform_iterator(
        zip, thrust::make_zip_function(thrust::not_equal_to{}));
    return 1 + thrust::reduce(map,
                              map + (last - first - 1),
                              0,
                              thrust::plus{});
}
```



```
template <class Derived,
          class ForwardIt,
          class BinaryPred>
typename thrust::iterator_traits<ForwardIt>::difference_type
_CCCL_HOST_DEVICE
unique_count(execution_policy<Derived> &policy,
             ForwardIt first,
             ForwardIt last,
             BinaryPred binary_pred)
{
    if (first == last) {
        return 0;
    }
    auto size = thrust::distance(first, last);
    auto it    = thrust::make_zip_iterator(thrust::make_tuple(first, thrust::next(first)));
    return 1 + thrust::count_if(policy, it, thrust::next(it, size - 1),
                               zip_adj_not_predicate<BinaryPred>{binary_pred});
}
```



```
template <typename I>
auto unique_count_zip_count_if(I first, I last) {
    auto zip = thrust::make_zip_iterator(first, first + 1);
    auto neq = thrust::make_zip_function(thrust::not_equal_to{});
    return 1 + thrust::count_if(zip, zip + (last - first - 1), neq);
}
```



```
auto unique_count(auto data) {  
    return data.map_adj(parrot::neq{}).sum() + 1;  
}
```



```
auto unique_count(auto data) {  
    return data.differ().sum() + 1;  
}
```



`thrust::transform_reduce`

`transform_iterator + reduce`

★ `map + reduce`

`thrust::unique_count`

`zip_iterator + count_if`

★ `map_adj + sum`

`thrust::tabulate`

`counting_iterator +
transform`

`range + ★ map`

[[end of digression]]



```
// https://github.com/PaddlePaddle/Paddle/blob/80f1123eb0c...
```

```
template <typename T>
static void Get_____(/* ... */) {
    // initialization

    for (int64_t i = 0; i < num_rows; ++i) {
        T* begin = input_tmp_data + num_cols * i;
        T* end = input_tmp_data + num_cols * (i + 1);
        thrust::device_vector<int64_t> indices_data(num_cols);
        thrust::sequence(thrust::device,
            indices_data.begin(),
            indices_data.begin() + num_cols);
        thrust::sort_by_key(thrust::device, begin, end, indices_data.begin());
        int unique = thrust::unique_count(thrust::device, begin, end);
        thrust::device_vector<T> keys_data(unique);
        thrust::device_vector<int64_t> cnts_data(unique);
        thrust::reduce_by_key(thrust::device,
            begin,
            end,
            thrust::constant_iterator<int>(1),
            keys_data.begin(),
            cnts_data.begin());
        auto it = thrust::max_element(
            thrust::device, cnts_data.begin(), cnts_data.begin() + unique);
        T ____ = keys_data[it - cnts_data.begin()];
        int64_t counts = cnts_data[it - cnts_data.begin()];
        auto pos = thrust::find(thrust::device, begin, end, mode);
        int64_t index = indices_data[pos - begin + counts - 1];
        out_tensor_ptr[i] = static_cast<T>(mode);
        indices_tensor_ptr[i] = static_cast<int64_t>(index);
    }
}
```




```
// https://github.com/PaddlePaddle/Paddle/blob/80f1123eb0c...
```

```
template <typename T>
static void Get_____(/* ... */) {
    // initialization

    for (int64_t i = 0; i < num_rows; ++i) {
        T* begin          = input_tmp_data + num_cols * i;
        T* end            = input_tmp_data + num_cols * (i + 1);
        auto indices_data = thrust::device_vector<int64_t>(num_cols);

        thrust::sequence(indices_data.begin(), indices_data.begin() + num_cols);
        thrust::sort_by_key(begin, end, indices_data.begin());

        int unique      = thrust::unique_count(thrust::device, begin, end);
        auto keys_data   = thrust::device_vector<T>(unique);
        auto cnts_data   = thrust::device_vector<int64_t>(unique);

        thrust::reduce_by_key(
            begin,
            end,
            thrust::constant_iterator<int>(1),
            keys_data.begin(),
            cnts_data.begin());

        auto it = thrust::max_element(cnts_data.begin(), cnts_data.begin() + unique);

        T ____          = keys_data[it - cnts_data.begin()];
        int64_t counts    = cnts_data[it - cnts_data.begin()];
        auto pos          = thrust::find(begin, end, mode);
        int64_t index     = indices_data[pos - begin + counts - 1];
        out_tensor_ptr[i] = static_cast<T>(mode);
        indices_tensor_ptr[i] = static_cast<int64_t>(index);
    }
}
```



```
template <typename Array>
auto GetModeBySort_Parrot(
    const Array& data, int num_rows, int num_cols) {
    using T = typename Array::value_type;
    std::vector<thrust::pair<T, int>> results;

    for (int r = 0; r < num_rows; ++r) {
        auto mode = parrot::stats::mode(data.row(r)).value();
        auto index = data.row(r).last_index_of(mode);
        results.push_back(thrust::make_pair(mode, index));
    }

    return parrot::array(results);
}
```



```
template <typename Array>
auto GetModeBySort_Parrot(
    const Array& data, int num_rows, int num_cols) {
    using T = typename Array::value_type;
    std::vector<thrust::pair<T, int>> results;

    for (int r = 0; r < num_rows; ++r) {
        auto mode = data.row(r)
            .sort()
            .rle()
            .max_by_key(parrot::snd())
            .value();
        auto index = data.row(r).last_index_of(mode);
        results.push_back(thrust::make_pair(mode, index));
    }

    return parrot::array(results);
}
```

Sushi for Two







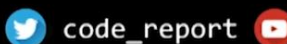


Conor Hoekstra



New Algorithms in C++23

Conor Hoekstra

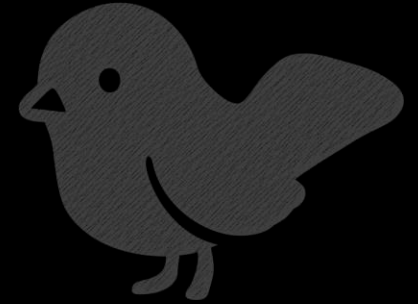


cppnorth.ca

<https://www.youtube.com/watch?v=VZPKHqeUQqQ>



```
template <int N>
constexpr auto sushi_for_two(std::array<int, N> sushi) {
    int current_sushi      = 0;
    int sushi_in_a_row     = 0;
    int prev_sushi_in_a_row = 0;
    int max_of_mins        = 0;
    for (auto const s : sushi) {
        if (current_sushi != s) {
            current_sushi = s;
            if (prev_sushi_in_a_row == 0) {
                prev_sushi_in_a_row = sushi_in_a_row;
                sushi_in_a_row      = 1;
            } else {
                auto const min      = std::min(sushi_in_a_row, prev_sushi_in_a_row);
                max_of_mins         = std::max(max_of_mins, min);
                prev_sushi_in_a_row = sushi_in_a_row;
                sushi_in_a_row      = 1;
            }
        } else {
            sushi_in_a_row += 1;
        }
    }
    auto const min = std::min(sushi_in_a_row, prev_sushi_in_a_row);
    max_of_mins    = std::max(max_of_mins, min);
    return max_of_mins * 2;
}
```



```
using namespace ranges::views;
using namespace combinators;

auto sushi_for_two(std::vector<int> sushi) {
    auto indices = concat(                                     //
        concat(single(0),                                     //
            zip(zip_with(_neq_, sushi, sushi | drop(1)),      //
                iota(1))                                       //
            | filter(_fst)                                     //
            | values),                                         //
        single(sushi.size()));                               //
    auto deltas = zip_with(_sub_, indices | drop(1), indices);
    return 2 * ranges::max(zip_with(_min_, deltas, deltas | drop(1)));
}
```



```
auto sushi_for_two(auto sushi) {  
    return sushi.differ()  
        .where()  
        .prepend(0)  
        .append(sushi.size())  
        .deltas()  
        .map_adj(parrot::min{})  
        .dbl()  
        .maxr();  
}
```



```
auto sushi_for_two(auto sushi) {  
    return sushi.differ()  
        .where() // 🔥  
        .prepend(0)  
        .append(sushi.size())  
        .deltas()  
        .map_adj(parrot::min{})  
        .dbl()  
        .maxr(); // 🔥  
}
```

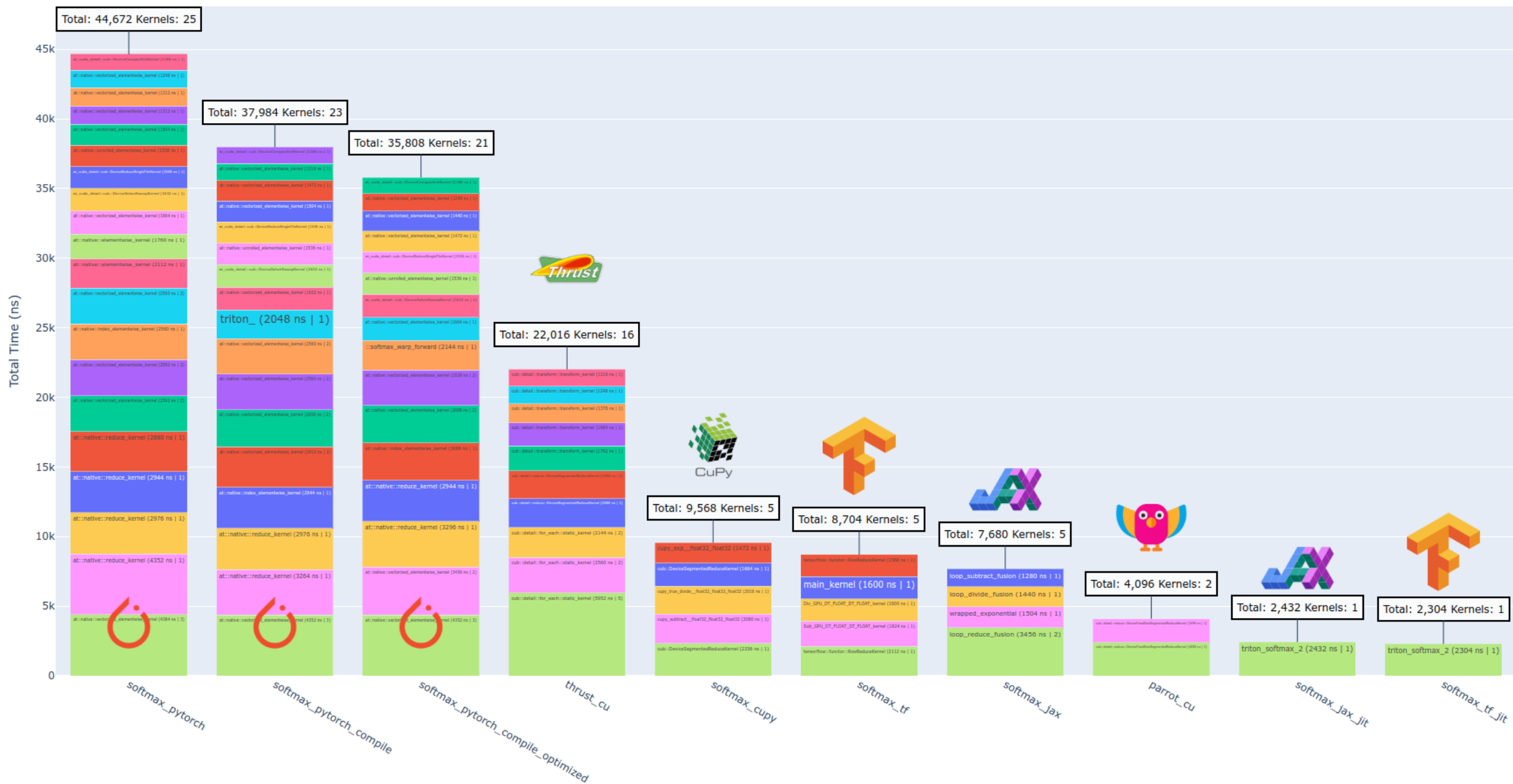
Softmax



```
int main() {  
    auto matrix = // ... (matrix initialization)  
  
    auto cols = matrix.shape()[1];  
    auto z     = matrix - matrix.maxr<2>().replicate(cols);  
    auto num   = z.exp();  
    auto den   = num.sum<2>();  
    (num / den.replicate(cols)).print();  
  
    return 0;  
}
```

kp

CUDA Kernel Profiling (softmax)





Parrot

A high level, parallel, array-based library
with implicit fusion



Parrot

<https://github.com/nvlabs/parrot>
Apache 2.0 License



Parrot

Feedback / Questions?